

classes

CSCI
134

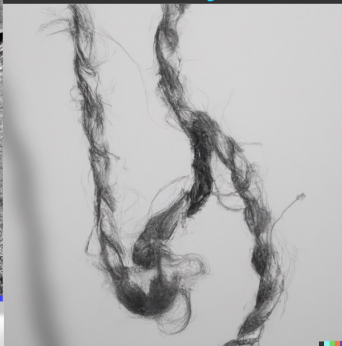
let's take a nostalgic
look back at the
eleven **expression types**
we've learned so far

type	example
int	7
float	3.14
str	"williams"
bool	3 > 4
function	abs
NoneType	print("hi")
range	range(1,5)
list	["mh", "rb", "jb", "rb"]
dict	{"a": 1, "b": 2, "c": 3}
set	{"mh", "rb", "jb"}
tuple	("mh", "rb", "jb", "rb")

range ↘



str ↘



float ↗

↗
NoneType

type

example

int

7

float

3.14

str

"williams"

bool

3 > 4

function

abs

NoneType

print("hi")

range

range(1,5)

list

["mh", "rb", "jb", "rb"]

dict

{"a": 1, "b": 2, "c": 3}

set

{"mh", "rb", "jb"}

tuple

("mh", "rb", "jb", "rb")

good times.

but what if we want

even more

types?

type	example
int	7
float	3.14
str	"williams"
bool	3 > 4
function	abs
NoneType	print("hi")
range	range(1,5)
list	["mh", "rb", "jb", "rb"]
dict	{"a":1, "b":2, "c":3}
set	{"mh", "rb", "jb"}
tuple	("mh", "rb", "jb", "rb")

good news.
python provides a
way

type	example
int	7
float	3.14
str	"williams"
bool	3 > 4
function	abs
NoneType	print("hi")
range	range(1,5)
list	["mh", "rb", "jb", "rb"]
dict	{"a":1, "b":2, "c":3}
set	{"mh", "rb", "jb"}
tuple	("mh", "rb", "jb", "rb")

MyNewType ???

back, for the third time

let's go ↓ to
vegas



previously
we created a
craps game



```
from random import randint

def roll_two_dice():
    die1 = randint(1, 6) # roll the first die
    die2 = randint(1, 6) # roll the second die
    total = die1 + die2 # compute the dice total
    print("You rolled a " + str(total))
    return total

def play_craps():
    dice_total = roll_two_dice()
    if dice_total in [7, 11]:
        print("... you win!")
    elif dice_total in [2, 3, 12]:
        print("... you lose!")
    else: # start phase two
        point = dice_total
        dice_total = roll_two_dice()
        while dice_total not in [7, point]:
            dice_total = roll_two_dice()
        if dice_total == point:
            print("... you win!")
        else:
            print("... you lose!")
```

but we did it
without a type
dedicated to **dice**



```
from random import randint

def roll_two_dice():
    die1 = randint(1, 6) # roll the first die
    die2 = randint(1, 6) # roll the second die
    total = die1 + die2 # compute the dice total
    print("You rolled a " + str(total))
    return total

def play_craps():
    dice_total = roll_two_dice()
    if dice_total in [7, 11]:
        print("... you win!")
    elif dice_total in [2, 3, 12]:
        print("... you lose!")
    else: # start phase two
        point = dice_total
        dice_total = roll_two_dice()
        while dice_total not in [7, point]:
            dice_total = roll_two_dice()
        if dice_total == point:
            print("... you win!")
        else:
            print("... you lose!")
```



```
from random import randint

class Die:
    def roll(die):
        return randint(1, 6)
```

Code

```
In [1]: from dice import Die

In [2]: my_die = Die()

In [3]: Die.roll(my_die)
Out[3]: 6

In [4]: my_die.roll()
Out[4]: 2
```

IPython

tells python:
we want a new
type called **Die**



```
from random import randint  
  
class Die:  
    def roll(die):  
        return randint(1, 6)
```



Code

```
In [1]: from dice import Die  
  
In [2]: my_die = Die()  
  
In [3]: Die.roll(my_die)  
Out[3]: 6  
  
In [4]: my_die.roll()  
Out[4]: 2
```

IPython

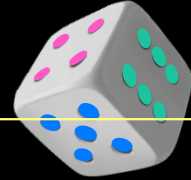
the **methods** are
indented beneath
the class declaration

```
from random import randint
```

```
class Die:
```

```
    def roll(die):
```

```
        return randint(1, 6)
```



Code

```
In [1]: from dice import Die
```

```
In [2]: my_die = Die()
```

```
In [3]: Die.roll(my_die)
```

```
Out[3]: 6
```

```
In [4]: my_die.roll()
```

```
Out[4]: 2
```

IPython

we can call the methods
like ordinary functions
in the **Die** namespace
(not common)

```
from random import randint

class Die:
    def roll(die):
        return randint(1, 6)
```



Code

```
In [1]: from dice import Die
```

```
In [2]: my_die = Die()
```

```
In [3]: Die.roll(my_die)
```

```
Out[3]: 6
```

```
In [4]: my_die.roll()
```

```
Out[4]: 2
```

iPython



or we can use
the preferred
shorthand

```
from random import randint

class Die:
    def roll(die):
        return randint(1, 6)
```



Code

```
In [1]: from dice import Die
```

```
In [2]: my_die = Die()
```

```
In [3]: Die.roll(my_die)
```

```
Out[3]: 6
```

```
In [4]: my_die.roll()
```

```
Out[4]: 2
```

iPython

recall:

the **First argument** of a method is always an expression of that type

```
In [1]: str.strip("line from a file\n")
Out[1]: 'line from a file'

In [2]: str.split("alpha,bravo,charlie", ",")
Out[2]: ['alpha', 'bravo', 'charlie']

In [3]: list.count([5, 1, 2, 5, 0], 5)
Out[3]: 2

In [4]: list.index([5, 1, 2, 5, 0], 0)
Out[4]: 4
```

Diagram illustrating the first argument of a method call:

- `str.strip()`: The first argument is `"line from a file\n"`, which is a `str` object.
- `str.split()`: The first argument is `"alpha,bravo,charlie"`, which is a `str` object.
- `list.count()`: The first argument is `[5, 1, 2, 5, 0]`, which is a `list` object.
- `list.index()`: The first argument is `[5, 1, 2, 5, 0]`, which is a `list` object.

Arrows labeled "is a" indicate the type of the first argument in each case.

```
from random import randint
```

```
class Die:
```

```
    def roll(die):
```

```
        return randint(1, 6)
```



Code

```
In [1]: from dice import Die
```

```
In [2]: my_die = Die()
```

```
In [3]: Die.roll(my_die)
```

```
Out[3]: 6
```

```
In [4]: my_die.roll()
```

```
Out[4]: 2
```

iPython

```
from random import randint

class Die:
    def roll(die):
        return randint(1, 6)
```

Code

class (or type):
the abstract
concept

```
In [1]: from dice import Die
```

```
In [2]: die1 = Die()
```

```
In [3]: die2 = Die()
```

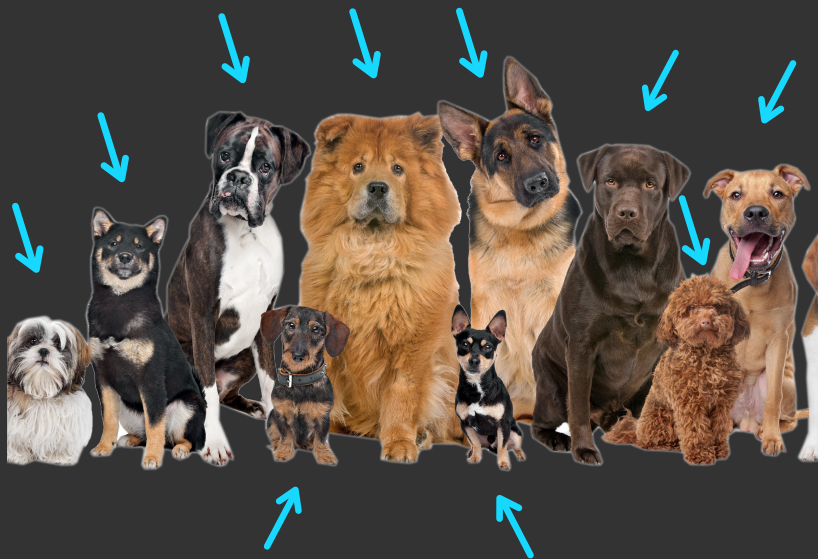
```
In [4]: die1 == die2
```

```
Out[4]: False
```

iPython

object:
a specific
instance

dog



class (or type):
the abstract
concept

object:
a specific
instance

since the **First argument** of a method is always an object of that type, it is typically referred to as **self**

```
from random import randint

class Die:
    def roll(die):
        return randint(1, 6)

    def roll_k_times(die, k):
        return [die.roll() for i in range(k)]
```

Code

```
from random import randint

class Die:
    def roll(self):
        return randint(1, 6)

    def roll_k_times(self, k):
        return [self.roll() for i in range(k)]
```

Code

a revised craps game



changes

```
from random import randint
```

```
class Die:
```

```
    def roll(self):  
        | return randint(1, 6)
```

```
def roll_dice(dice):
```

```
    total = 0  
    for die in dice:  
        | total += die.roll()  
    print("You rolled a " + str(total))  
    return total
```

```
def play_craps(dice):
```

```
    dice_total = roll_dice(dice)  
    if dice_total in [7, 11]:  
        | print("... you win!")  
    elif dice_total in [2, 3, 12]:  
        | print("... you lose!")  
    else: # start phase two  
        point = dice_total  
        dice_total = roll_dice(dice)  
        while dice_total not in [7, point]:  
            | dice_total = roll_dice(dice)  
        if dice_total == point:  
            | print("... you win!")  
        else:  
            | print("... you lose!")
```

now we can change
the game just by
changing the behavior
of the **dice**

```
from random import randint
```

```
class Die:
```

```
    def roll(self):  
        | return randint(1, 6)
```

```
def roll_dice(dice):  
    total = 0  
    for die in dice:  
        | total += die.roll()  
    print("You rolled a " + str(total))  
    return total
```

```
def play_craps(dice):  
    dice_total = roll_dice(dice)  
    if dice_total in [7, 11]:  
        | print("... you win!")  
    elif dice_total in [2, 3, 12]:  
        | print("... you lose!")  
    else: # start phase two  
        point = dice_total  
        dice_total = roll_dice(dice)  
        while dice_total not in [7, point]:  
            | dice_total = roll_dice(dice)  
        if dice_total == point:  
            | print("... you win!")  
        else:  
            | print("... you lose!")
```

```
from random import randint
```

```
class Die:
```

```
    def roll(self):  
        | return randint(1, 6)
```

becomes

```
def roll_dice(dice):
```

```
    total = 0  
    for die in dice:  
        | total += die.roll()  
    print("You rolled a " + str(total))  
    return total
```

```
def play_craps(dice):
```

```
    dice_total = roll_dice(dice)  
    if dice_total in [7, 11]:  
        | print("... you win!")  
    elif dice_total in [2, 3, 12]:  
        | print("... you lose!")  
    else: # start phase two  
        point = dice_total  
        dice_total = roll_dice(dice)  
        while dice_total not in [7, point]:  
            | dice_total = roll_dice(dice)  
        if dice_total == point:  
            | print("... you win!")  
        else:  
            | print("... you lose!")
```

```
from random import choice
```

```
class Die:
```

```
    def roll(self):  
        | return choice([1,2,3,4,5,5,5,6,6,6])
```

```
def roll_dice(dice):
```

```
    total = 0  
    for die in dice:  
        | total += die.roll()  
    print("You rolled a " + str(total))  
    return total
```

```
def play_craps(dice):
```

```
    dice_total = roll_dice(dice)  
    if dice_total in [7, 11]:  
        | print("... you win!")  
    elif dice_total in [2, 3, 12]:  
        | print("... you lose!")  
    else: # start phase two  
        point = dice_total  
        dice_total = roll_dice(dice)  
        while dice_total not in [7, point]:  
            | dice_total = roll_dice(dice)  
        if dice_total == point:  
            | print("... you win!")  
        else:  
            | print("... you lose!")
```

now we have
loaded
dice

